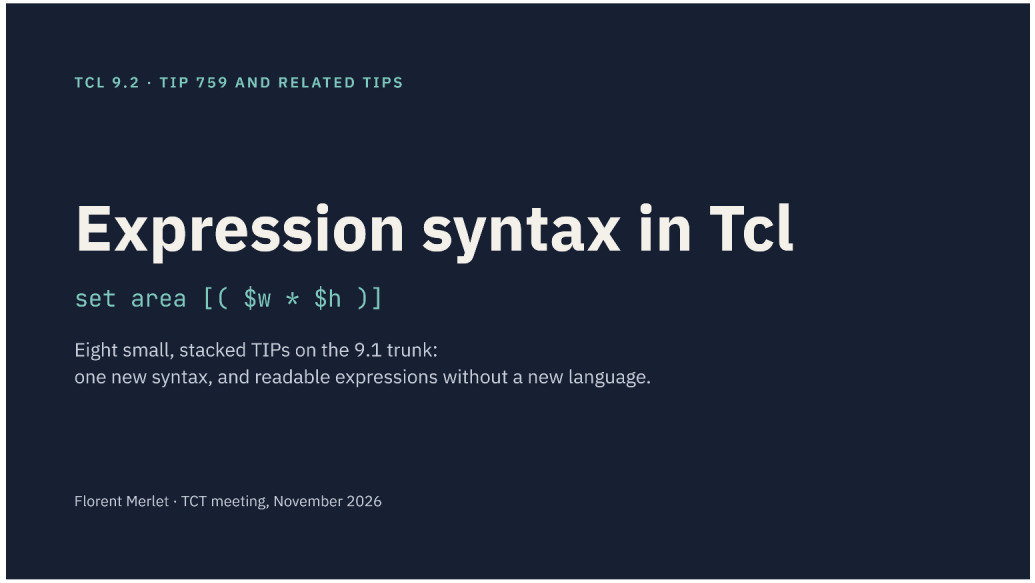


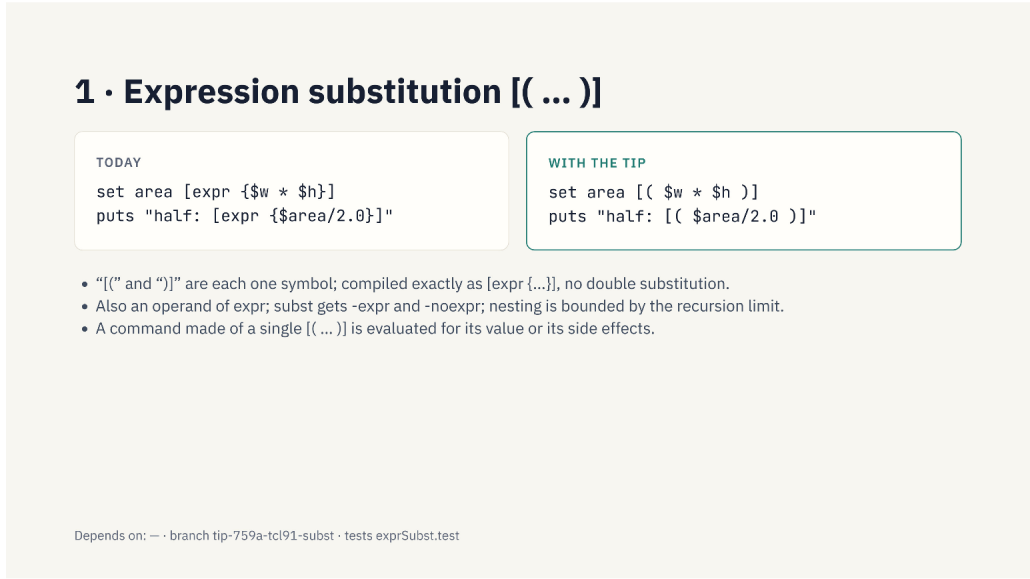
TIP 759 was one large proposal. After the discussion on the forum, it is now split into eight independent patches, each with its tests, its documentation and its own TIP draft. All of them are on core.tcl-lang.org as branches tip-759a to tip-759h.

Eight TIPs, one per patch

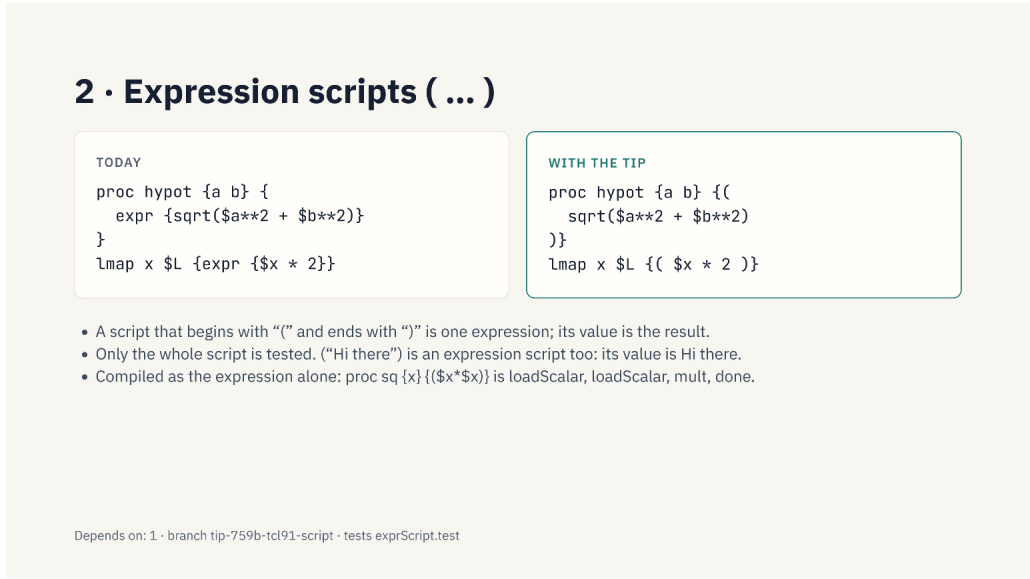
#	TIP	Example	Depends on
1	759 · Expression substitution	[($w * $h)]	—
2	Expression scripts	proc sq {x} {($x*$x)}	1
3	Lists in expressions	($x, $y) · [(1,2),(3,4)]	—
4	282 · Assignment and “;”	x = 5*$y; $x + 1	3, for “;” in lists
5	Tuple assignment	(a, (b, c)) = $v	3, 4
6	Raw commands (provisional)	llength $L + 1	4
7	Expansion {*}	max({*}$L)	3, for list elements
8	User-defined operators	$U \dot $V	—

Stacked on the 9.1 trunk, applied in order · branches tip-759a to tip-759h on core.tcl-lang.org

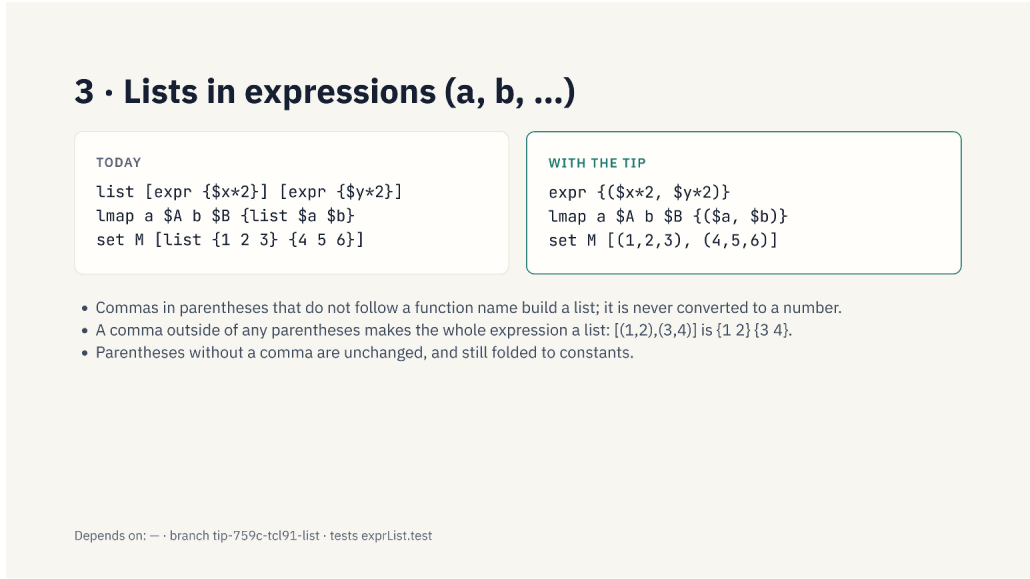
Each patch has its own tests, its documentation and a TIP draft. They are stacked: each branch starts from the previous one. The real dependencies are fewer than the chain: lists and TIP 282 do not need the expression substitution.



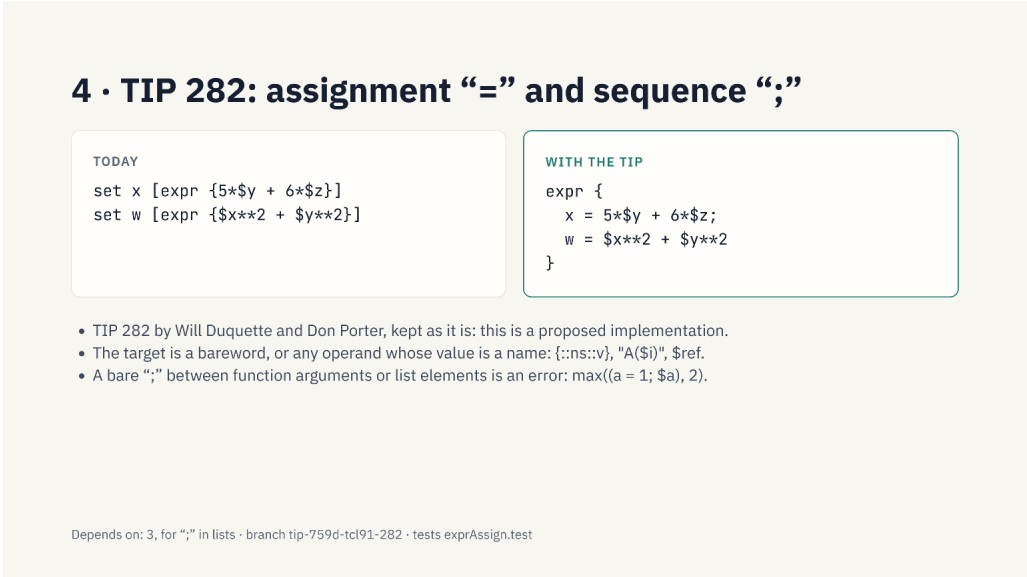
This TIP is the base the others build on, and can be accepted on its own. The only incompatibility: a command whose name begins with an open parenthesis, called right after an open bracket. It can still be called with a space after the bracket. On the way, the port fixed two bugs of the 9.0 branch: expr {[(1)]+1} gave 1, and subst {[(1 +)]} looped forever.



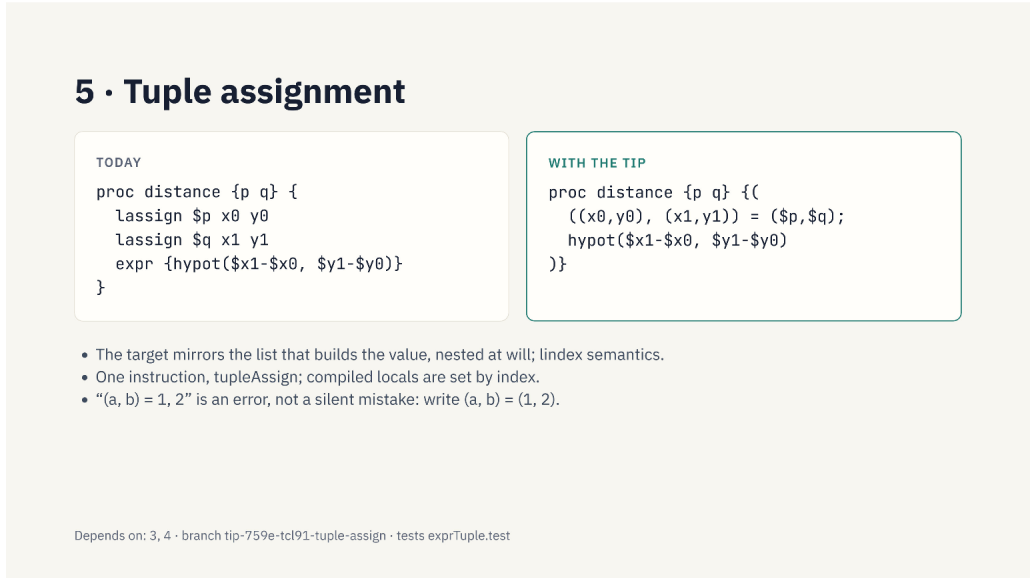
Incompatibility: a script that is exactly a call of a command whose name begins with an open paren and whose last argument ends with a close paren, such as (cmd) (x). A space before the first paren keeps the old meaning. Open point: braces and parens are read as one symbol, so no white space between them.



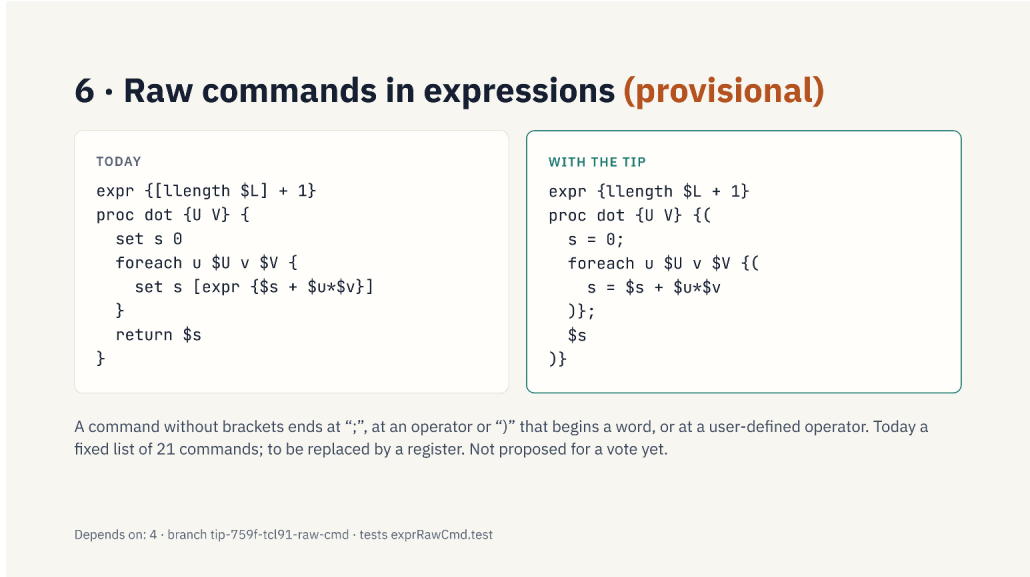
Only expressions that are errors today change meaning: expr {1,2} was “unexpected comma outside function argument list”. The parentheses of an expression substitution may close early, as in [(1)+(2)], which gives the matrix form [(1,2),(3,4)]. Internally the START node becomes START_LIST: no node is added, and the list is built with the list instruction instead of tryCvtToNumeric.



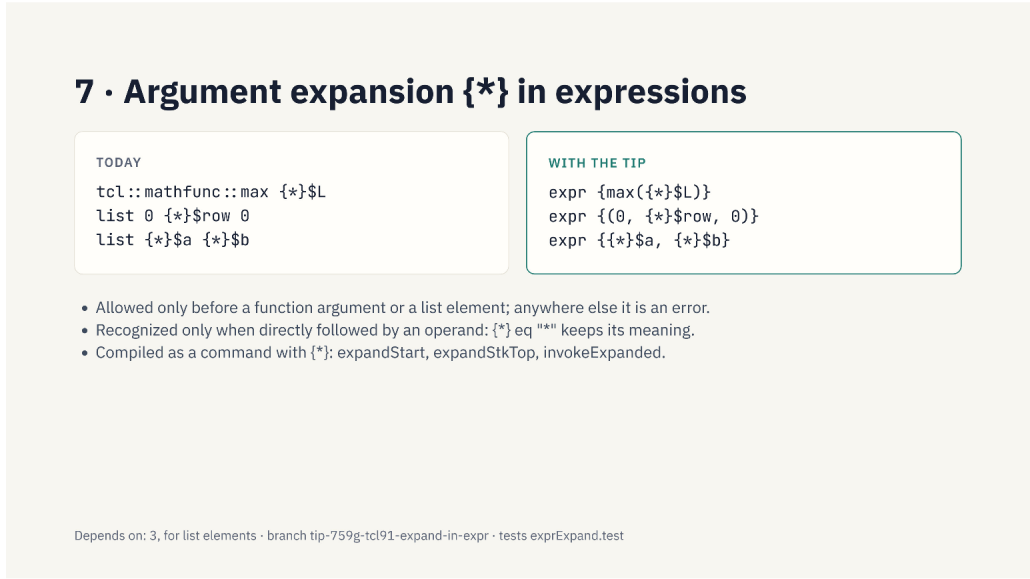
In a procedure, an assignment to a compiled local compiles to storeScalar. “=” groups right to left and binds looser than “?:”. Empty statements have the empty string as value. Points for the authors: the free target (2 = 3 is valid and sets a variable named 2), and the rule on bare semicolons, which avoids (1, 2; 3) silently giving 1 3.



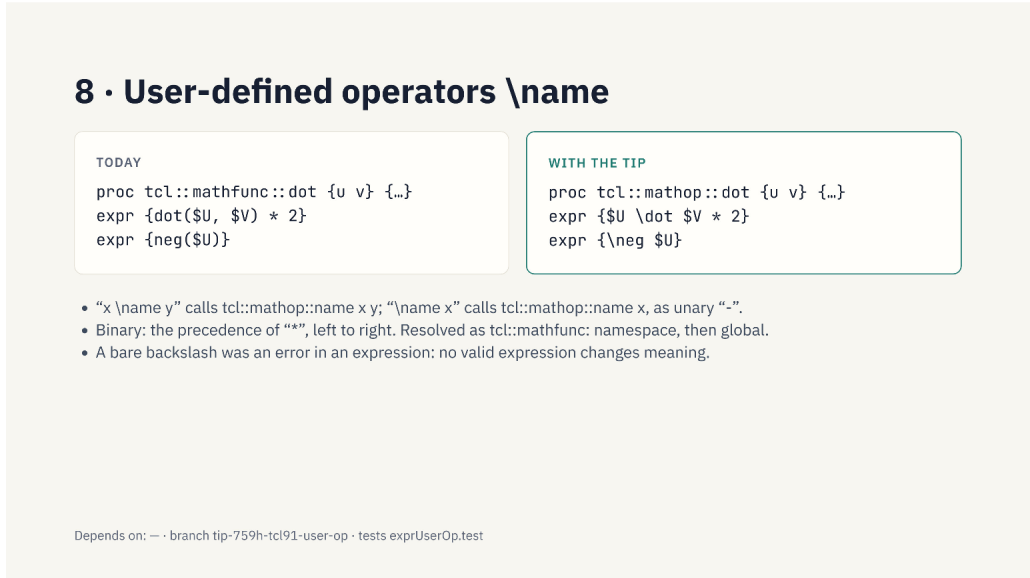
A missing element gives the empty string, extra elements are ignored, as with lassign. A name may be given by any operand: (a, $n) = … sets the variable named by n. The error on a tuple among list elements came with the lists outside parentheses: the comma binds looser than “=”, so (a, b) = 1, 2 would assign 1 to the tuple and leave b empty.



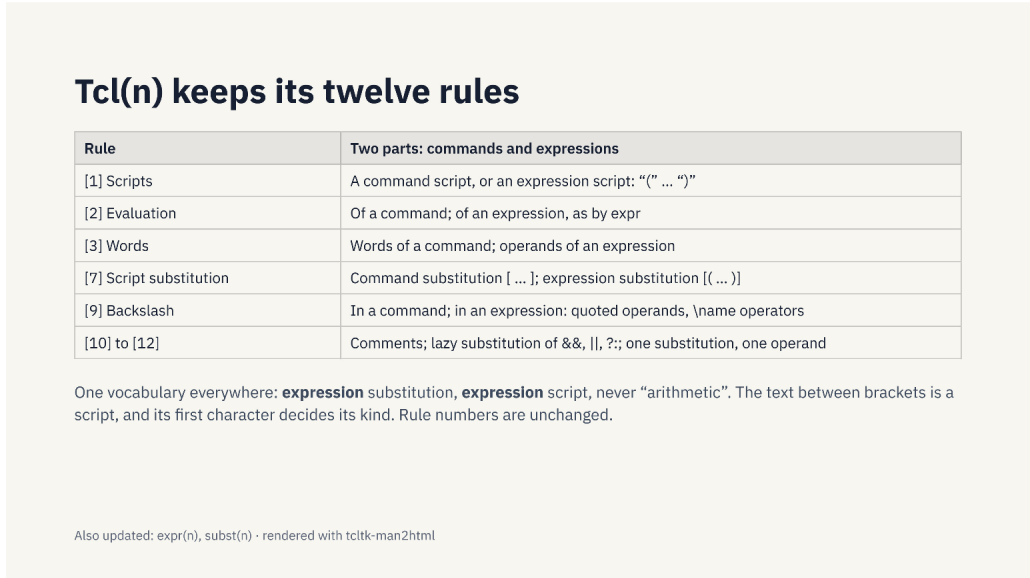
This part shows where the series is heading: procedure bodies written as one expression script. The fixed list of commands is the weak point. The plan is a register of the commands allowed without brackets, designed together with a register of the variables known to expr, for example "A is array", which would let A(k) name an array element without quotes.



Without {*}, the bytecode of an expression does not change. An empty list adds nothing. Future work: {*} could mark the rest of a tuple, (first, {*}rest) = $L, as lassign does; it is an error today, so it can be added later.



The precedence is fixed because bytecode is compiled once and cached, and the command may be defined later. Limit: the operator is read by the expression parser, so it works in braces, in [( )] and in expression scripts; in expr "..." the Tcl parser first replaces backslash sequences such as \t in \times. A declaration such as “\dot is operator” could come later with the register of names known to expr.



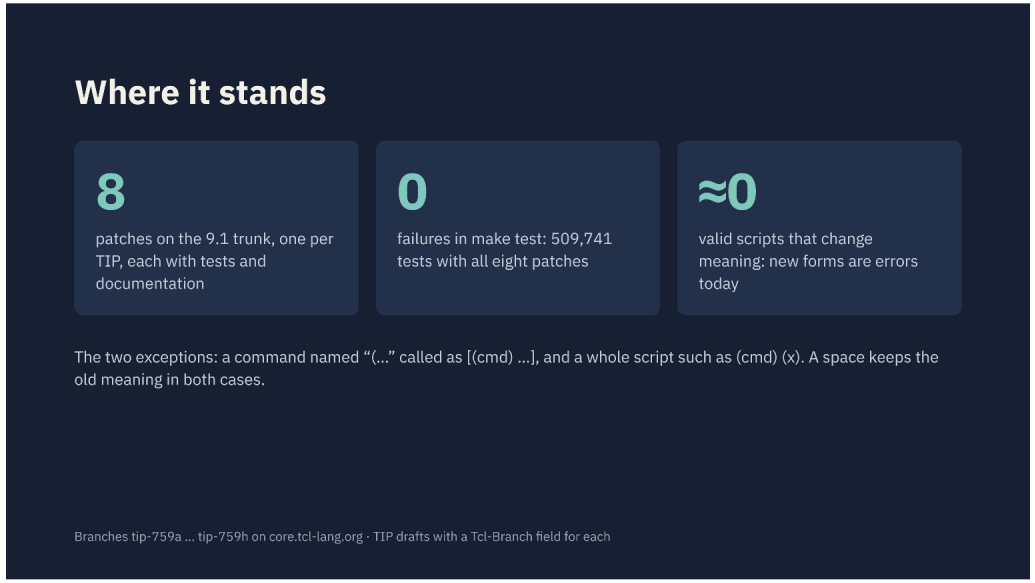
Tcl(n) keeps its twelve rules

Rule	Two parts: commands and expressions
[1] Scripts	A command script, or an expression script: “(” … “)”
[2] Evaluation	Of a command; of an expression, as by expr
[3] Words	Words of a command; operands of an expression
[7] Script substitution	Command substitution […]; expression substitution [(…)]
[9] Backslash	In a command; in an expression: quoted operands, \name operators
[10] to [12]	Comments; lazy substitution of &&, ||, ?:; one substitution, one operand

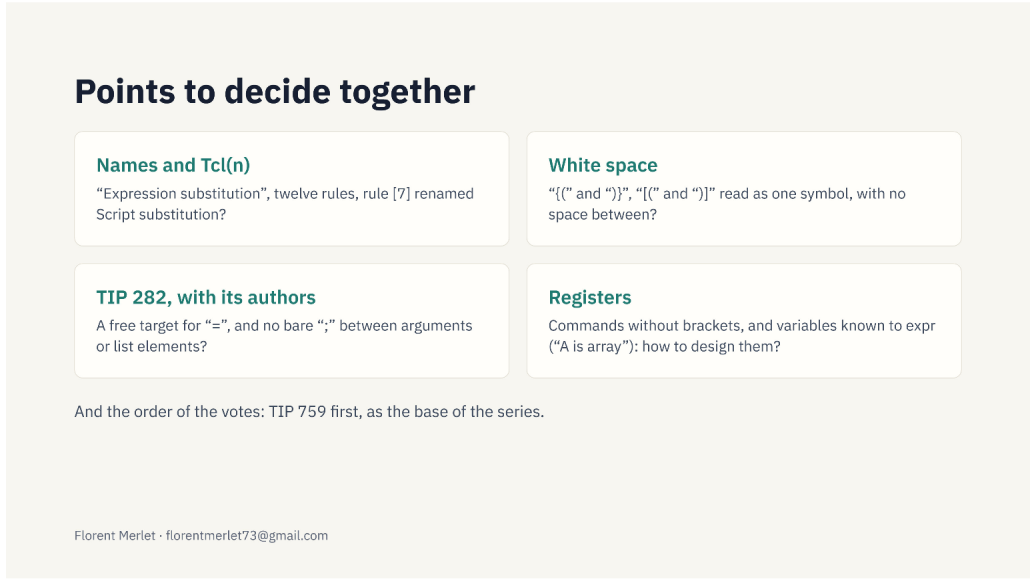
One vocabulary everywhere: **expression** substitution, **expression** script, never “arithmetic”. The text between brackets is a script, and its first character decides its kind. Rule numbers are unchanged.

Also updated: expr(n), subst(n) · rendered with tcltk-man2html

This answers the review: [( )] is not an extension of command substitution but a sibling of it, under rule [7] renamed Script substitution. Two other layouts were considered: ten rules with all substitutions grouped, or thirteen rules with a new rule; both renumber rules cited everywhere. An expression script such as (“Hi there”) is valid: its value is the string.



The numbers are from the full test suite on Linux, on the fossil trunk of 30 October, with all eight patches applied. New test files: exprSubst, exprScript, exprList, exprAssign, exprTuple, exprRawCmd, exprExpand, exprUserOp. A handful of existing tests changed because they expected errors that are now valid forms, for example expr 0,0.



Suggested order of votes: 759 (expression substitution), then expression scripts, lists, TIP 282 with Will Duquette and Don Porter, tuples, expansion and user-defined operators. Raw commands wait for the register.